

VAL CoPilot

Azure AI Foundry Deployment Guide

Step-by-step procedure to provision the LangChain cognitive routing agent into Microsoft Azure AI Foundry using **copilot_agent/deploy_to_foundry.py**.

Audience: Azure Cloud Architects / DevSecOps Engineers

Script: `copilot_agent/deploy_to_foundry.py`

SDKs: `azure-ai-projects 1.0.0` · `azure-ai-agents` · `azure-identity`

1. Overview

This guide deploys VAL CoPilot as a **managed Azure AI Foundry Agent**. The deployment script authenticates with Entra ID, wraps Fabric SQL and Azure AI Search MCP tools in a Foundry **FunctionTool / ToolSet**, and creates the agent using the existing multi-agent **SYSTEM_PROMPT** (compliance audits, red-flagging, financial exposure projections, and renewal strategy sheets).

What gets provisioned

- **Managed agent** named `val-copilot` (configurable)
- **Model deployment** referenced by `AZURE_OPENAI_DEPLOYMENT_NAME` or `AZURE_FOUNDRY_MODEL_DEPLOYMENT`
- **ToolSet** containing: `get_expiring_contracts`, `get_vendor_spend_summary`, `search_cloud_blob_contracts`
- **Instructions** = full `SYSTEM_PROMPT` from `copilot_agent/agent.py`

2. Prerequisites

2.1 Azure resources

- 1 An Azure subscription with permission to create / manage AI resources.
- 2 A **Microsoft Foundry project** (foundry-based, not hub-only) with Agents enabled.
- 3 A **model deployment** in that project (for example `gpt-4o`).
- 4 Microsoft Fabric SQL Gold layer reachable via `ActiveDirectoryDefault` (for tool runtime).
- 5 Azure AI Search index with contract documents (for semantic search tool).

2.2 Local workstation

- 1 Python 3.10+ (3.12 recommended) and a virtual environment.
- 2 Azure CLI installed; identity able to use `DefaultAzureCredential`.
- 3 Git clone of the VALAgent repository.
- 4 Network path to Fabric, Azure AI Search, and the Foundry project endpoint.

2.3 Entra ID / RBAC

Assign the deploying identity (user or service principal) at least **Azure AI Developer** (or equivalent) on the Foundry project, plus access to Fabric and Azure AI Search data planes used by the wrapped tools.

3. Gather Foundry Connection Information

In the Azure AI Foundry portal, open your **Project Overview** page and copy the **Project endpoint**. Preferred form:

```
https://<ai-services-account>.services.ai.azure.com/api/projects/<project-name>;
```

Alternatively, use a legacy semicolon-delimited connection string:

```
endpoint=https://<account>.services.ai.azure.com/api/projects/<project>;subscriptionId=<guid>;resourceGroupName=<rg>;projectName=<project>;
```

*Note: azure-ai-projects 2.x removed connection-string factories. This repo pins **azure-ai-projects==1.0.0** so ToolSet / FunctionTool remain available; the script parses `AZURE_FOUNDRY_CONNECTION_STRING` and initializes `AIProjectClient` from the endpoint.*

4. Configure Root Environment (.env)

Copy `.env.example` to `.env` at the repository root (do not create per-folder env files). Set at minimum:

Variable	Required	Purpose
AZURE_FOUNDRY_CONNECTION_STRING	Yes	Foundry project endpoint or legacy connection string
AZURE_OPENAI_DEPLOYMENT_NAME or AZURE_FOUNDRY_MODEL_DEPLOYMENT	Yes	Model deployment name used by the managed agent
AZURE_FOUNDRY_AGENT_NAME	No	Agent display name (default: val-copilot)
FABRIC_SQL_SERVER / FABRIC_SQL_DATABASE	Runtime	Fabric Gold SQL for expiring contracts & spend tools
AZURE_SEARCH_ENDPOINT / API_KEY / INDEX	Runtime	Azure AI Search for contract document tool

```
# Example fragment for root .env
AZURE_FOUNDRY_CONNECTION_STRING=https://myacct.services.ai.azure.com/api/projects/val
AZURE_FOUNDRY_MODEL_DEPLOYMENT=gpt-4o
AZURE_FOUNDRY_AGENT_NAME=val-copilot
AZURE_OPENAI_DEPLOYMENT_NAME=gpt-4o
```

5. Authenticate with Azure (DefaultAzureCredential)

The script uses **DefaultAzureCredential** only (no API keys for Foundry control plane). Choose one interactive or automated path:

- 1 **Developer workstation:** run `az login` (and `az account set --subscription <id>` if needed).
- 2 **Service principal:** export `AZURE_CLIENT_ID`, `AZURE_TENANT_ID`, `AZURE_CLIENT_SECRET` (or use federated credentials / managed identity in CI).
- 3 **Cloud / VM:** attach a managed identity with Foundry + data-plane RBAC.

```
az login
az account show --query id -o tsv
```

6. Install Python Dependencies

From the repository root, create/activate a virtual environment for the orchestrator package and install requirements (includes pinned Foundry SDKs):

```
cd copilot_agent
python3 -m venv .env
source .env/bin/activate
pip install -r requirements.txt
pip install -r ../mcp_server/requirements.txt
```

Pinned packages: azure-ai-projects==1.0.0, azure-ai-agents>=1.0.0,<2.0.0, azure-identity>=1.19.0. Do not upgrade to azure-ai-projects 2.x for this script — ToolSet was removed.

7. Validate Wiring (Dry Run)

Before calling Azure, validate connection-string parsing, MCP tool import, ToolSet construction, and SYSTEM_PROMPT binding:

```
cd copilot_agent
python deploy_to_foundry.py --dry-run -v
```

Expected JSON highlights

- status: dry_run
- tools list includes get_expiring_contracts, get_vendor_spend_summary, search_cloud_blob_contracts
- non-zero instructions_chars (full SYSTEM_PROMPT loaded)
- endpoint / subscription / project fields parsed from AZURE_FOUNDRY_CONNECTION_STRING

8. Deploy the Managed Agent

When dry-run succeeds and Azure connectivity is available, provision the agent:

```
cd copilot_agent
python deploy_to_foundry.py -v
# optional name override:
python deploy_to_foundry.py --agent-name val-copilot-prod
```

What the script does at runtime

- 1 Load credentials via DefaultAzureCredential.
- 2 Parse AZURE_FOUNDRY_CONNECTION_STRING and open AIProjectClient.
- 3 Import MCP tools from mcp_server/server.py.
- 4 Wrap tools with FunctionTool(...) and ToolSet.add(...).
- 5 Call project_client.agents.create_agent(...) with model, name, instructions=SYSTEM_PROMPT, and toolset.
- 6 Print a JSON summary including agent_id on success.

9. Verify in Azure AI Foundry Portal

- 1 Open Azure AI Foundry → your project → **Agents**.
- 2 Confirm agent val-copilot (or custom name) exists.
- 3 Inspect Instructions — should contain Comparative Analysis and Contract Lifecycle Procedures.
- 4 Inspect Tools — three function tools for Fabric / Search operations.
- 5 Use the portal Playground (or SDK thread/run APIs) to ask: “Show expiring contracts” / “Red-flag compliance audit for CON-0003”.

10. Runtime & Architecture Notes

The Foundry-managed agent hosts the **cognitive instructions and tool definitions**. Function tools execute in the process that handles tool calls (this deployment path wraps the existing Python MCP implementations). Ensure the execution host can reach Fabric SQL (ODBC Driver 18 + ActiveDirectoryDefault) and Azure AI Search.

Cognitive procedures encoded in SYSTEM_PROMPT

- Comparative Analysis & Decision Framework → ## Recommendation

- Red-Flag Compliance Audits → ## Red-Flag Compliance Audit
- Dynamic Counter-Clause Drafting → ## Dynamic Counter-Clause Drafting
- Financial Exposure Projections → ## Financial Exposure Projection
- Proactive Renewal Strategy Sheets → ## Proactive Renewal Strategy Sheet

11. Troubleshooting

Symptom	Likely cause / fix
Missing AZURE_FOUNDRY_CONNECTION_STRING	Set variable in root .env; re-run from repo that loads config.py
Cannot import ToolSet / FunctionTool	Reinstall pinned azure-ai-projects==1.0.0 and azure-ai-agents<2
DefaultAzureCredential failed	Run az login or configure SP / managed identity env vars
403 / firewall to Azure OpenAI or Foundry	Allowlist egress IPs; confirm Agents enabled in project region
Tool returns empty / SQL errors	Validate Fabric + Search env vars; test MCP server locally first
Wrong model name	Match AZURE_FOUNDRY_MODEL_DEPLOYMENT to an existing project deployment

12. Deployment Checklist (Quick Reference)

- Foundry project endpoint copied
- Model deployed in the project
- Root .env populated (Foundry + Fabric + Search)
- az login / managed identity ready
- pip install -r copilot_agent/requirements.txt (+ mcp_server deps)
- python deploy_to_foundry.py --dry-run succeeds
- python deploy_to_foundry.py returns status=created with agent_id
- Portal Agents view shows tools + SYSTEM_PROMPT procedures
- Smoke-test expiring contracts / compliance audit prompts

Document generated for VALAgent · Script path: copilot_agent/deploy_to_foundry.py · Keep secrets out of Git; commit only .env.example.